STAT3612 – Group 10

Brain Tumour Classification
Group Project

Guo Yihan, Huang Xu, Zhong Tianheng

April 23, 2026

Contribution Table

Name	UID	Contribution	% of Contribution
Huang Xu	3036325042	Algorithm Research, Model Development	33%
Guo Yihan	3036418112	Background Research, Statistical Analysis	33%
Zhong Tianheng	3036296863	Model Development, Report	33%

Content

Abstract 3

1. Introduction

1.1. Background Information 3

1.2. Related Research 3

1.3. Aims and Objectives 4

2. Methodology

2.1. Data Preprocessing 5

2.2. Hypothesis Testing 6

2.3. Feature Engineering 7

2.4. TF-IDF Vectorization 8

2.5. Model 9

2.6. Bayesian Hyperparameter Optimization 10

2.7. Training & Evaluation 11

2.8. Model Interpretability 12

3. Results

3.1. Classification Report 14

3.2. ROC Curve 14

3.3. Overall Assessment & Kaggle 15

4. Discussion & Evaluation

4.1. Why do some methods or models fail? 16

4.2. Why do we add feature augmentation? 17

4.3. Overall assessment of our model 17

5. Conclusion 18

References 19

Appendix 20

Abstract

This project develops a hybrid machine learning pipeline for multiclass classification of primary brain tumors (Glioma, Meningioma, Brain Metastasis, Sellar region, Pineal/-Choroid plexus) using structured clinical data and free-text radiology reports. The methodology integrates: (1) dual TF-IDF vectorization capturing word-level semantics and character-level morphology; (2) domain-informed feature engineering including anatomical keywords and clinical indicators; (3) ANOVA-filtered radiomics features with missingness handling; (4) Bayesian hyperparameter optimization of LightGBM; and (5) targeted upsampling of minority classes.

The final model achieves a Kaggle public micro-F_1 score of 0.7432, demonstrating that a hybrid text-structure approach with careful imbalance handling effectively supports presurgical tumor classification from routinely collected clinical data.

1 Introduction

1.1 Background Information

Primary brain and central nervous system (CNS) tumors are among the most clinically challenging categories in oncology, encompassing a heterogeneous spectrum of histological subtypes with widely divergent prognoses and treatment pathways. Epidemiological data from the CBTRUS Statistical Report (2017–2021) estimate an age-adjusted annual incidence of 25.34 per 100,000 in the United States population, with malignant tumors—including glioblastoma and medulloblastoma—contributing disproportionately to mortality [Price et al., 2024]. Brain and CNS cancers remain the leading cause of cancer-related death in children aged 0-14, underscoring the urgent need for accurate, timely, and accessible diagnostic tools.

Historically, definitive tumor classification has relied on histopathological and molecular analyses of surgically obtained tissue, which are invasive, costly, and often unavailable in resource-constrained settings. The increasing availability of structured clinical records and radiology reports in modern hospital systems presents an underutilized opportunity: leveraging routinely collected data to support or accelerate tumor classification through machine learning, thereby reducing dependence on time-intensive pathological workflows.

1.2 Related Research

The conceptual foundation for extracting diagnostic signals from medical imaging data was articulated by Gillies et al. [Gillies et al., 2016], who defined radiomics as the high-throughput conversion of medical images into mineable, high-dimensional feature sets. This paradigm showed that quantitative descriptors—encompassing intensity statistics, texture matrices, and shape indices—could capture intra-tumoral heterogeneity invisible to qualitative review, enabling diagnostic and prognostic models built entirely on standard-of-care acquisitions. However, early radiomics research suffered from a critical

reproducibility gap: feature values were inconsistent across software implementations, limiting multi-site generalizability. The Image Biomarker Standardization Initiative (IBSI) addressed this directly [Zwanenburg et al., 2020], establishing reference values for 169 of 174 standardized features across CT, PET, and MRI modalities through a multi-phase consensus involving 25 independent teams. The resulting framework made reproducible, comparable radiomic pipelines a practical reality rather than an aspirational goal.

Building on these standardization efforts, Wang et al. [Wang et al., 2024] demonstrated that AI-enabled MRI analysis could replace invasive molecular testing for medulloblastoma subgrouping. Their pipeline—combining deep learning segmentation, radiomic feature extraction, and an ensemble of gradient-boosted and neural classifiers—achieved AUCs exceeding 0.90 in prospective consecutive validation across a 934-patient, 13-center international cohort [Wang et al., 2024]. This work validated that machine learning models can generalize across racially and geographically diverse populations when trained on sufficiently large, standardized datasets.

Parallel advances in natural language processing have shown that unstructured radiology reports contain rich diagnostic signals amenable to automated classification. TF-IDF vectorization, particularly when augmented with character-level n-grams and domain-engineered binary indicators, provides a computationally efficient and interpretable representation of clinical narratives [Pedregosa et al., 2011, Ramos, 2003]. When combined with structured tabular features in a gradient-boosted tree framework such as LightGBM [Ke et al., 2017], this hybrid approach has proven particularly effective for heterogeneous, class-imbalanced medical datasets, where deep neural alternatives suffer from instability due to limited support for the minority class.

1.3 Aims & Objectives

This project develops and evaluates a machine learning pipeline for the multiclass classification of primary brain and CNS tumors into five clinically meaningful categories—Glioma, Meningioma, Brain Metastasis Tumor, Tumors of the Sellar Region, and Pineal/Choroid Plexus Tumor—using structured clinical variables and free-text radiology reports.

The primary objective is to construct a robust, computationally efficient classification system that achieves high micro-F_1 performance across all classes, including those with severe class imbalance. To this end, the pipeline integrates dual TF-IDF representations (word and character n-grams), domain-informed feature engineering, Bayesian hyperparameter optimization, and minority-class upsampling within a stratified cross-validation framework.

A secondary objective is to assess the comparative advantages of gradient boosting over neural-network baselines in this specific data regime, thereby providing a principled justification for the final model selection.

2 Methodology

2.1 Data Preprocessing

The raw dataset comprises three complementary information sources that are merged at the patient level prior to any modeling. The first source consists of JSON case files, each containing a free-text radiology report, a tumor subtype label, and metadata indicating available imaging modalities. The second source is an optional clinical CSV file containing structured patient-level variables such as age, sex, and relevant clinical indicators. A third source — modality-wise radiomics feature files — is merged in a subsequent step. For each patient, the observed tuple is formally:

$$\left(X_i^{\text{clinical}}, X_i^{\text{radiomics}}, X_i^{\text{report}}, Y_i\right)$$

where $Y_i \in \{1, \ldots, 5\}$ denotes the tumor subtype. The JSON structure is converted into a DataFrame and left-joined on `case_id`, producing a single patient-level row per example. This design ensures the classification problem is treated as independent and identically distributed at the patient level, rather than at the token or modality level.

Radiomics columns undergo a dedicated cleaning pipeline before being admitted to the feature matrix. First, columns with a missing rate exceeding a defined threshold are dropped entirely. For retained radiomics columns, binary missingness indicator features are appended, allowing the model to distinguish informative absence from true zero values. A central preprocessing challenge is severe class imbalance. If n_k denotes the number of training examples in class k, the imbalance ratio:

$$r = \frac{\max_k n_k}{\min_k n_k}$$

is large, particularly for the Pineal/Choroid plexus and Sellar-region classes, which have only a handful of examples relative to Glioma and Meningioma. Because the evaluation metric is micro-F_1, which weights all classes equally, focus classes are selectively upsampled within training folds toward:

$$\tilde{n}_k = \max(3n_k, \text{median class count})$$

applied strictly within training folds to prevent data leakage into validation. This partial rebalancing pushes minority classes away from the extreme tail without distorting the overall class distribution. All preprocessing transformers—vectorizers, scalers, and encoders—are fitted exclusively on training data and applied to validation and test sets, ensuring no distributional information leaks across splits.

Table 1 summarizes the class distribution across train and validation splits and highlights a strong long-tail pattern. Glioma and Meningioma dominate the sample, while Sellar-region and Pineal/Choroid plexus classes remain extremely sparse, which justifies class-rebalancing and micro-F_1-oriented optimization.

Table 1: Class Distribution (Train/Val)

Class	Train	Val	Total	Train %	Val %	Total %
Glioma	924	132	1056	46.60	46.64	46.60
Meningioma	728	104	832	36.71	36.75	36.72
Brain Metastase Tumour	252	36	288	12.71	12.72	12.71
Tumors of the sellar region	56	8	64	2.82	2.83	2.82
Pineal tumour and Choroid plexus tumour	23	3	26	1.16	1.06	1.15

The train-split imbalance ratio is $\max_k n_k / \min_k n_k = 924/23 \approx 40.17$, which motivates class-rebalancing during model training.

2.2 Hypothesis Testing

Prior to feature construction, statistical hypothesis testing is applied as a principled filter over the radiomics feature space. Because radiomics pipelines routinely produce hundreds of candidate descriptors, retaining all features without selection introduces noise dimensions that inflate model variance without contributing discriminative signal, a problem particularly acute for minority classes with small support.

For radiomics features specifically, a univariate one-way ANOVA F-test is used to screen each candidate descriptor for class-discriminative signal. Let x_j denote the radiomics feature j and y denote the class label. For each feature, the ANOVA F-statistic is computed as

$$F_j = \frac{\text{between class variance of } x_j}{\text{within class variance of } x_j}$$

then converted to a p-value, and the feature is retained only when:

$$p_j \leq 0.05$$

This screening is implemented in `select_radiomics_by_anova` in `src/lgbm.py` (for full code, please check Appendix) and applied consistently within each cross-validation fold to prevent information leakage from test-fold label distributions.

Table 2: ANOVA Filter Summary

Train n	Val n	Total radiomics	Selected ($p \leq 0.05$)	Retention
1983	283	20	14	70.0%

Table 2 depicts the filtered summary. In the current fixed-parameter reference run, 14 of 20 candidate radiomics features survived this filter, yielding a retention ratio of 70.0%.

Table 3: Top 5 ANOVA Features

Feature	F	p-value
`ax_t1c__rad_firstorder_Mean`	9.615	1.06×10^{-7}
`ax_t1c__rad_firstorder_90Percentile`	8.919	3.89×10^{-7}
`ax_t1__rad_glcm_Contrast`	7.346	7.17×10^{-6}
`ax_t2f__rad_firstorder_90Percentile`	7.261	8.40×10^{-6}
`ax_t1__rad_glcm_JointEntropy`	4.994	5.26×10^{-4}

Table 3 indicates that several top-ranked features come from `axial_T1c/T2f` first-order intensity and `T1` texture statistics, suggesting these modalities capture the strongest class-discriminative radiomic signals in this dataset.

The retained features are subsequently passed through missingness-based filtering, with binary missingness indicator columns appended to any retained radiomics column that contains missing values, thereby stabilizing radiomics signals across splits. For binary domain-indicator features derived from the radiology report, a chi-squared test of independence is used to assess whether the proportion of positive cases differs significantly across tumor subtypes, serving as a complementary filter for the engineered feature block.

2.3 Feature Engineering

Beyond raw text representation, the pipeline engineers a set of domain-informed features by mapping the radiology report through deterministic transformations:

$$\phi_j : X^{\text{report}} \to \mathbb{R}^m$$

where each component $\phi_j(x)$ is a binary indicator, a count, or a multiplicative interaction term. Examples include: `has_hydrocephalus`, `has_enhancement`, `has_pineal`, `has_sellar`, `has_ventricular`, and `has_extra_axial_meningioma_keywords`, defined formally as:

$$\phi_{\text{hydro}}(x) = \mathbf{1}\big\{\text{“hydrocephalus”} \in x\big\}$$

with interaction terms constructed as products, for example,

$$\phi_{\text{enh}\times\text{edema}}(x) = \phi_{\text{enh}}(x) \cdot \phi_{\text{edema}}(x)$$

These features are motivated by the observation that minority classes are characterized by highly localized anatomical language: Pineal and Choroid plexus tumors are frequently associated with ventricular obstruction and pineal-region localization, while sellar-region tumors are linked to “sellar”, “suprasellar”, and “pituitary”. Injecting this domain knowledge as low-variance binary signals provides the model with stable minority-class discriminators that purely learned representations may miss at this dataset size, consistent with the approach of Wang et al. [Wang et al., 2024]. Additional lightweight report statistics — document length, word count, unique-word ratio,

digit count, and punctuation count — are appended as structural descriptors. ANOVA-filtered radiomics descriptors, together with their associated missingness indicators, are concatenated into the tabular block, so that the full structured representation is:

$$x_i^{\text{tab}} = \left[x_i^{\text{clinical}}, x_i^{\text{domain}}, x_i^{\text{radiomics}}, x_i^{\text{miss}}\right]$$

```
df["report"] = df["report"].fillna("").str.lower()
df["has_hydrocephalus"] = df["report"].str.contains("hydrocephalus").
    astype(int)
df["has_pineal"] = df["report"].str.contains("pineal").astype(int)
df["has_sellar"] = df["report"].str.contains(r"sellar|suprasellar|
    pituitary").astype(int)
df["focus_keyword_hits"] = df["has_hydrocephalus"] + df["has_pineal"
    ] + df["has_sellar"]
```

The code snippet above shows how raw report text is mapped to compact binary clinical indicators. These indicators inject stable, domain-specific signals that are especially useful for minority classes.

2.4 TF-IDF Vectorization

The radiology report is the dominant information source in this task and is represented using Term Frequency-Inverse Document Frequency (TF-IDF) vectorization. For a term t in the document d, the TF-IDF weight is:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log\left(\frac{N}{d \cdot f(t)}\right)$$

where $\text{tf}(t, d)$ is term frequency, $d \cdot f(t)$ is document frequency, and N is the total number of documents. TF-IDF provides a computationally efficient and interpretable sparse projection of clinical narratives into a linear feature space, well-suited to this task because the dataset is not large enough to fully exploit the sample demands of deep language models [Pedregosa et al., 2011, Ramos, 2003].

Two complementary text views are constructed in parallel. First, a word-level TF-IDF vectorizer uses unigrams and bigrams (range 1-2) with a vocabulary of up to 2,500 features and a minimum document frequency of 2. Second, a character-level TF-IDF vectorizer operates over character window n-grams of width 3-5, producing up to 1,200 features. The two representations are concatenated into a single sparse matrix:

$$x_i^{\text{text}} = \left[x_i^{\text{char}}, x_i^{\text{word}}\right]$$

Word bigrams extract semantic medical phrases such as "midline shift", "sellar region", or "hydrocephalus", which carry a strong class-discriminative signal. Character n-grams provide robustness to lexical variation, abbreviations, and morphological inflections common in non-standardized radiology text. The full patient feature vector is then formed as:

$$x_i = \left[x_i^{\text{tab}}, x_i^{\text{text}}\right]$$

```
tfidf_word = TfidfVectorizer(ngram_range=(1, 2), min_df=2,
   max_features=2500)
tfidf_char = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5),
    min_df=2, max_features=1200)

X_word = tfidf_word.fit_transform(X_train["report"])
X_char = tfidf_char.fit_transform(X_train["report"])
X_text = hstack([X_word, X_char], format="csr")
X_train_encoded = hstack(
    [csr_matrix(X_train_tab_encoded.to_numpy()), X_text],
    format="csr"
)
```

The code snippet above builds two complementary sparse text views: word-level and character-level TF-IDF. The merged text matrix is then concatenated with tabular features to form the final model input.

2.5 Model

The classifier is a multiclass LightGBM gradient-boosted tree [Ke et al., 2017]. LightGBM grows trees in a leaf-wise manner, selecting at each step the leaf whose split maximally improves the training objective, which tends to produce stronger empirical performance for structured, heterogeneous feature spaces under a fixed computational budget. The model optimizes an additive ensemble objective

$$\mathcal{L} = \sum_{i=1}^{n} \ell\left(y_i, \hat{y}_i\right) + \sum_{m=1}^{K} \Omega(f_m)$$

where ℓ is the multiclass cross-entropy loss, each f_m is a decision tree, and Ω penalizes tree complexity. The final output for the patient i is a vector of class probabilities $\hat{p}_{ik} = \Pr(Y_i = k \mid x_i)$, with the predicted label taken as $\hat{y}_i = \arg\max_k \hat{p}_{ik}$. Hyperparameters are not fixed constants but are learned per run via Bayesian HPO, with the search space bounded as:

- `n_estimators` $\in [120, 800]$,
- `learning_rate` $\in [10^{-2.5}, 10^{-0.7}]$,
- `num_leaves` $\in [16, 128]$,
- and `max_depth` $\in [-1, 20]$, with regularization terms $\alpha, \lambda \in [10^{-6}, 10^{1}]$.

To compensate for class imbalance, the model uses `class_weight` set to "balanced", reweighting each class's contribution to the loss as $w_k \propto 1/\pi_k$, so that errors on rare classes incur heavier penalties. Following model training, a two-stage posterior class-scaling procedure is applied.

In the first stage, scales are tuned on the holdout validation split. In the second stage, out-of-fold (OOF) probabilities from 5-fold cross-validation on the full labeled set (train + val) are used to re-tune scales, and these OOF-derived scales are used as the default for final test inference.

$$\tilde{p}_{ik} = \frac{s_k \cdot \hat{p}_{ik}}{\sum_{j=1}^{K} s_j \cdot \hat{p}_{ij}}$$

where $s_k > 0$ is the tuned scale for class k. In `src/lgbm_fixed.py`, the fixed class-scale configuration is set to: Brain Metastase Tumour $s = 1.0$, Pineal tumour and Choroid plexus tumour $s = 3.0$, and Tumors of the sellar region $s = 1.9$. LightGBM is preferred over Random Forest because its sequential error correction via boosting is more expressive for the sparse multiclass boundary, and over a BPNN because the high-dimensional sparse input and small minority-class support make stochastic gradient updates unreliable at this dataset scale.

```
X_train_balanced, y_train_balanced = upsample_focus_classes(
   X_train_encoded, y_train_enc, focus_ids)

def apply_class_scales(proba, scales):
    p = proba * scales.reshape(1, -1)
    return p / p.sum(axis=1, keepdims=True)
```

The code snippet above first partially rebalances minority classes by targeted upsampling in the training set. It then rescales posterior probabilities so under-represented classes receive calibrated decision mass at inference time.

2.6 Bayesian Hyperparameter Optimization

Rather than performing grid or random search, the pipeline employs Bayesian optimization via a Gaussian process surrogate model implemented in `bayes_optimize`. The core idea is to model the unknown objective $f(\theta) = \text{score}(\theta)$ with a Gaussian process and select the next candidate by maximizing an upper-confidence-bound acquisition function:

$$a(\theta) = \mu(\theta) + 2\sigma(\theta)$$

where $\mu(\theta)$ is the surrogate's predictive mean and $\sigma(\theta)$ is its predictive uncertainty. To ensure numerical stability of the Gaussian process across runs, the collected objective values are standardized before fitting:

$$\tilde{y} = \frac{y - \bar{y}}{\max(\text{std}(y), \varepsilon)}, \quad \varepsilon = 10^{-8}$$

This z-scaling prevents the GP from becoming ill-conditioned when early trial scores cluster tightly. The upper-confidence-bound rule balances exploitation of currently high-performing regions with exploration of uncertain ones, making it substantially more sample-efficient than grid search for a mixed continuous-discrete hyperparameter space [Snoek et al., 2012].

Critically, the optimized objective is a composite blended score:

$$\text{Blend} = 0.45 \cdot F_1^{\text{weighted}} + 0.35 \cdot F_1^{\text{macro}} + 0.20 \cdot \text{Recall}_{\text{focus}}$$

where $\text{Recall}_{\text{focus}}$ averages recall over the most difficult minority classes. This ensures that the hyperparameter search directly rewards minority-class sensitivity rather than collapsing to majority-class accuracy.

We do not use Kaggle leaderboard scores (public/private micro-F_1) as the HPO objective. The private score is hidden during development, and repeatedly tuning against the public leaderboard risks leaderboard overfitting because it is computed on only a subset of the test set and can be noisy for rare classes. In contrast, the blended score is computed from local stratified validation data and is fully reproducible, making it a safer and more statistically stable optimization target.

```
# Final Bayesian-HPO parameters used in lgbm_fixed.py
FINAL_BAYES_PARAMS = {
    "n_estimators": 574, "learning_rate": 0.16151497813164442,
    "num_leaves": 43, "max_depth": 13, "min_child_samples": 25,
    "subsample": 0.8686242756333769,
    "colsample_bytree": 0.9578829278425386,
    "reg_alpha": 2.6062996462909116e-05,
    "reg_lambda": 0.11500730663980376,
    "objective": "multiclass", "class_weight": "balanced",
    "random_state": 42, "n_jobs": -1, "verbosity": -1,
}
```

2.7 Training and Evaluation

The full training workflow follows a sequential ten-step pipeline that enforces strict separation between tuning, validation diagnostics, and final test inference. Preprocessing transformers are fitted on training data only and applied to validation and test sets. Focus classes are upsampled within the training portion of each split. Five-fold stratified cross-validation diagnostics are then run on the combined train+val set, with fold-level HPO and fold-level scale tuning, to provide robustness monitoring across splits. Bayesian HPO is subsequently run on the main training/validation split to obtain `best_params`. A LightGBM model is fitted on the upsampled training set with these parameters, class scales are tuned on validation predictions, and OOF scales are re-tuned on train+val and set as the default for final inference. For the final test submission, a single model is trained on the full balanced train+val set with `best_params` and test predictions are produced using the OOF-tuned scales.

Evaluation uses accuracy, macro-F_1, weighted-F_1, and the blended HPO score as complementary metrics, with macro-F_1 treated as the primary indicator of class-balanced performance. The current fixed-parameter reference run (`lgbm_fixed.py`) reports a validation accuracy of 0.8799, macro-F_1 of 0.8075, weighted-F_1 of 0.8776, and a best blended score of 0.8477, with a Kaggle public score of 0.7432. The

pipeline is implemented across three aligned scripts: `src/lgbm_fixed.py` for the fixed-parameter production variant with radiomics filtering and 5-fold CV reporting, `src/lgbm.py` for the always-on Bayesian HPO workflow with OOF-driven scale tuning, and `src/stat3612.ipynb` for a portable notebook version of the same workflow.

2.8 Model Interpretability (SHAP)

To interpret the fixed-parameter LightGBM model, we compute TreeSHAP feature attributions using the same preprocessing pipeline and final parameter set used in `lgbm_fixed.py`. For each feature j, global importance is summarized by the mean absolute contribution:

$$I_j = \frac{1}{N} \sum_{i=1}^{N} |\phi_{ij}| \,,$$

where ϕ_{ij} is the SHAP value of feature j for sample i. This formulation is useful because it captures average contribution magnitude regardless of sign, enabling stable ranking across heterogeneous feature blocks (text, radiomics, and clinical variables). We therefore report three complementary views: class-wise attribution patterns, global top-ranked features, and group-level attribution share.

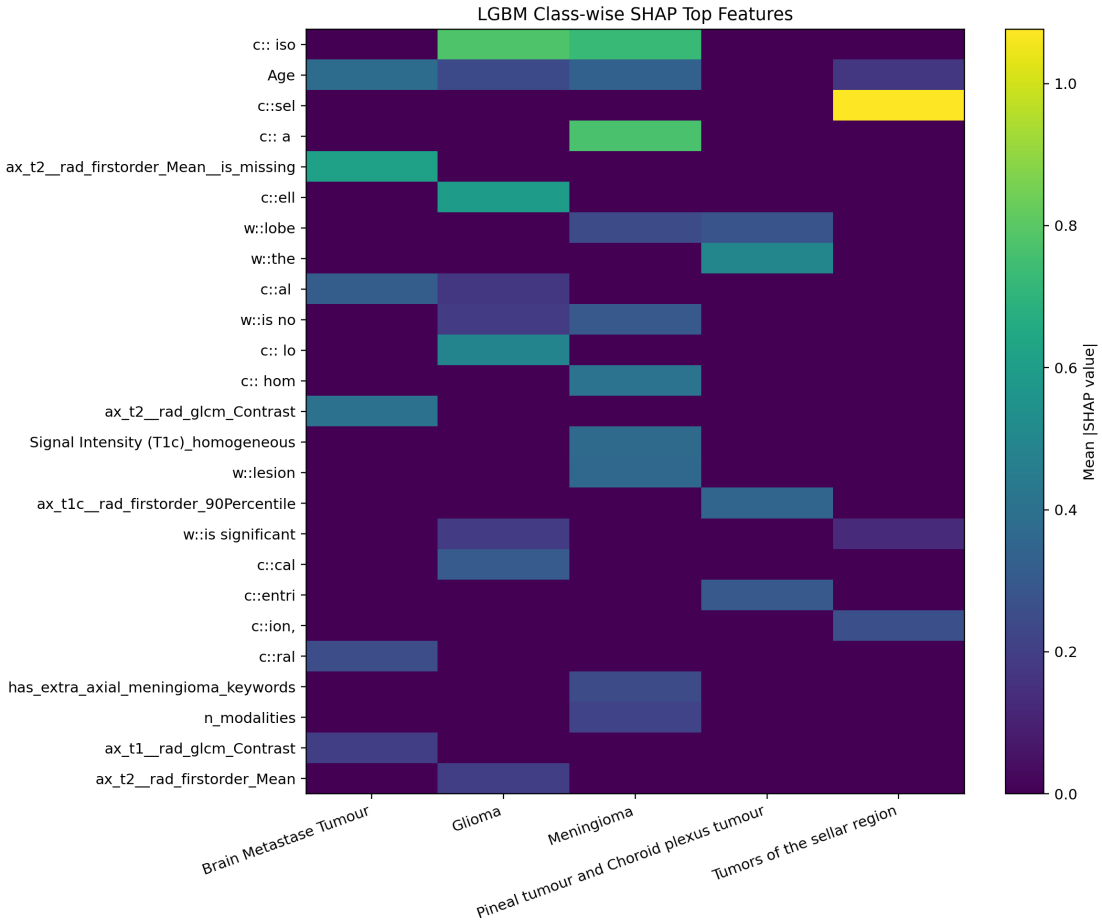

Figure 1: Class-wise SHAP heatmap of top 25 features for fixed-parameter LightGBM.

Figure 1 indicates clear class-specific structure rather than a single universal signal. Sellar-region predictions are strongly tied to sellar-related lexical tokens, while metastasis and glioma classes use a broader mixture of text and radiomics information; this pattern is consistent with known class-dependent language in neuroradiology reports. The heatmap also shows that some features are highly concentrated in one class, which supports the model's ability to separate minority classes instead of collapsing to majority-class decision boundaries.

Table 4: Top 10 Global SHAP Features (Fixed-Parameter LightGBM)

Rank	Feature	Mean |SHAP|
1	`c::iso`	0.308
2	`c::sel`	0.230
3	`Age`	0.228
4	`c::a`	0.193
5	`c::ell`	0.146
6	`ax_t2__rad_firstorder_Mean__is_missing`	0.146
7	`w::the`	0.145
8	`ax_t1c__rad_firstorder_90Percentile`	0.143
9	`w::lobe`	0.127
10	`c::lo`	0.125

Table 4 shows that global attribution is dominated by TF-IDF-derived text signals, followed by selected radiomics descriptors and a key clinical variable (`Age`), consistent with the hybrid text–tabular model design. Notably, both lexical fragments (for example, `c::sel`) and quantitative imaging descriptors (for example, `ax_t1c__rad_firstorder_90Percentile`) appear in the top 10, indicating that the model is not purely keyword-driven and still leverages imaging phenotypes.

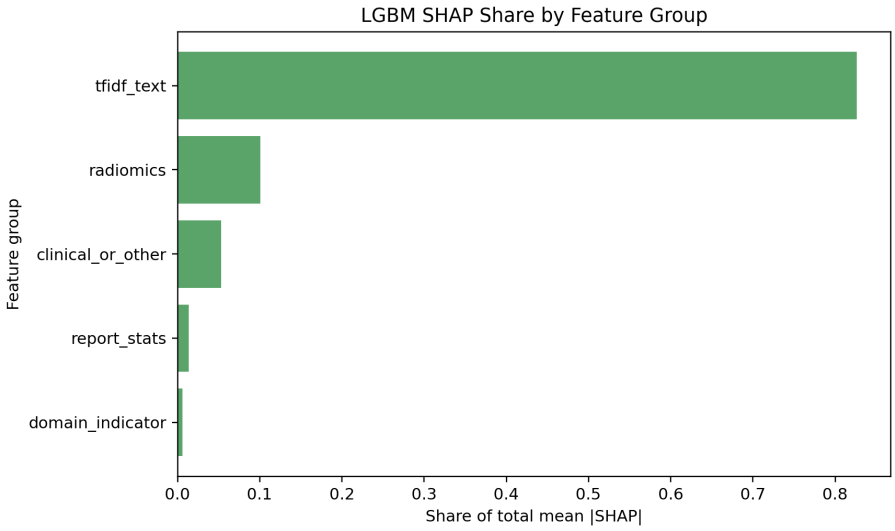

Figure 2: SHAP attribution share by feature group for fixed-parameter LightGBM.

Figure 2 summarizes attribution share across groups: TF-IDF features contribute approximately 82.64% of total attribution mass, radiomics contributes about 10.08%, and clinical/other structured features contribute about 5.28%, with report statistics and domain indicators forming smaller residual components. This distribution justifies the design choice of using text as the primary information channel while retaining radiomics and structured features as complementary signals that improve robustness and class specificity.

3 Results

3.1 Classification Report

Table 5 reports the validation-set classification performance of the fixed-parameter LightGBM model ($n = 283$). The model achieves an overall performance (accuracy $= 0.88$, macro-$F_1 = 0.81$, weighted-$F_1 = 0.88$), while still showing expected instability on the rarest classes due to extremely low support.

Table 5: Validation Classification Report (Fixed-Parameter LightGBM)

Class	Precision	Recall	F1-score	Support
Brain Metastase Tumour	0.74	0.64	0.69	36
Glioma	0.87	0.89	0.88	132
Meningioma	0.93	0.96	0.94	104
Pineal tumour and Choroid plexus tumour	0.67	0.67	0.67	3
Tumors of the sellar region	1.00	0.75	0.86	8
Accuracy	–	–	0.88	283
Macro average	0.84	0.78	0.81	283
Weighted average	0.88	0.88	0.88	283

The class-wise pattern is consistent with the long-tail distribution shown earlier: majority classes (Glioma, Meningioma) are predicted more stably, whereas minority classes exhibit larger variance in recall and F_1. The pineal/choroid class remains the hardest due to support $n = 3$, where one misclassification substantially shifts per-class metrics.

3.2 ROC Curve

Figure 3 presents one-vs-rest ROC curves on the validation split using scaled class probabilities from the fixed-parameter LightGBM pipeline, and shows that all class-specific curves lie well above the random baseline, indicating strong ranking ability across decision thresholds. The one-vs-rest AUC values are high for all five classes (Brain Metastase: 0.929, Glioma: 0.963, Meningioma: 0.989, Pineal/Choroid plexus: 0.992, Sellar region: 0.965), which suggests that the fixed-parameter LightGBM model produces well-separated probability distributions between positive and negative samples for each class.

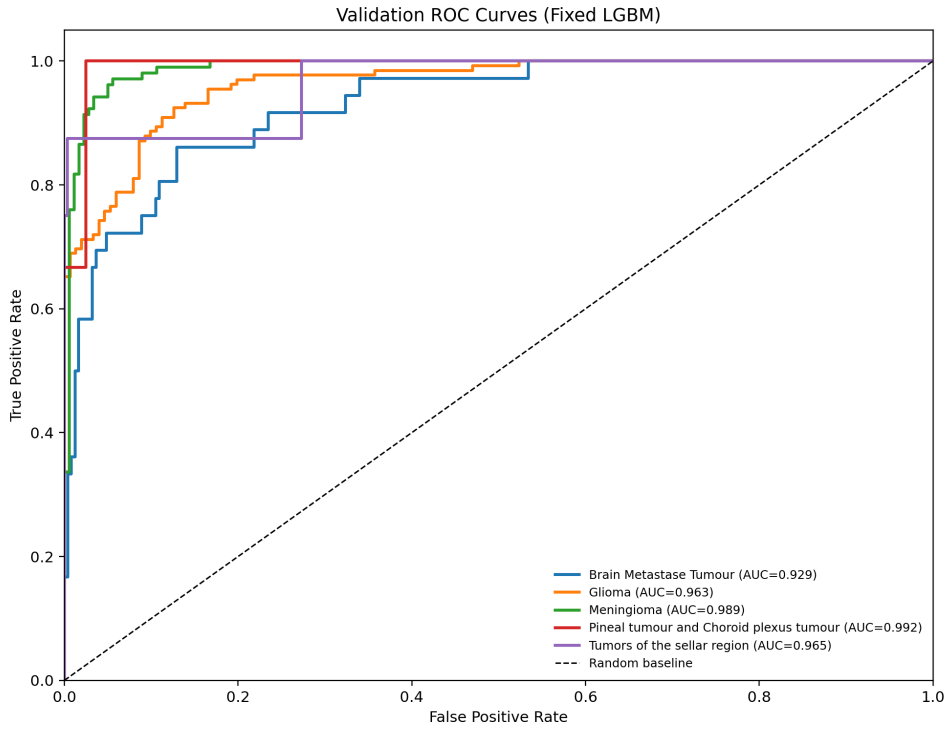

Figure 3: Validation ROC Curves (Fixed-Parameter LightGBM, one-vs-rest).

Notably, the Pineal/Choroid plexus and Sellar-region curves also achieve high AUC despite severe class imbalance. This indicates that class scaling and feature engineering improve ranking quality for minority classes. However, because these classes have very small validation support ($n = 3$ and $n = 8$), their ROC estimates can be statistically unstable; therefore, these AUC values should be interpreted together with the class-wise precision/recall results in Table 5 rather than in isolation.

3.3 Overall Assessment & Kaggle

Overall, the fixed-parameter LightGBM pipeline demonstrates strong and consistent performance for this multiclass clinical task. The combination of dual-view TF-IDF, domain-informed report indicators, ANOVA-filtered radiomics, and targeted minority-class handling yields robust validation performance (accuracy ≈ 0.88, weighted-$F_1 \approx 0.88$) while maintaining acceptable discrimination on under-represented classes. The SHAP analysis further supports that the model uses both lexical and radiomic signals rather than relying on a single feature source.

The main limitation remains class imbalance at the extreme tail, especially for Pineal/-Choroid plexus and Sellar-region tumors, where per-class metrics and ROC estimates can vary noticeably due to very small sample size. Hence, the current system is best interpreted as a high-performing and practical baseline, with future gains likely to come from additional minority-class data, calibrated threshold tuning, and external validation.

On the competition benchmark, the final submission from this pipeline achieves a Kaggle public micro-F_1 score of 0.7432. This result is consistent with our internal validation

trend and indicates good generalization from local splits to the held-out Kaggle test distribution. Because leaderboard scores can fluctuate with small probability shifts in rare classes, this public score should be viewed as supportive evidence rather than the sole criterion of model quality.

4 Discussion & Evaluation

4.1 Why do some methods or models fail?

Several alternative approaches failed to match our final LightGBM pipeline. The reasons reveal key insights about this task.

4.1.1 Forward Stepwise Feature Selection

Stepwise selection identified 14 "best" features achieving 0.6754 CV Macro-F_1. However, the validation performance did not generalize. The problem is that every feature contributes some signal. No single feature or small subset captures all discriminative information. By discarding 3,425 features, we lost weak but complementary signals that collectively matter.

4.1.2 Random Forest

Random Forest achieved excellent CV performance (0.7575 with 200 features) but collapsed to 0.69 on the public leaderboard—a severe generalization gap of nearly 0.07. The model overfit to our validation split. Random Forest is generally a strong baseline for mixed clinical variables [Breiman, 2001], but LightGBM holds concrete advantages here. First, unlike boosting, Random Forest reduces variance through bagging but lacks sequential error correction; once a tree misses a minority class example, no subsequent tree compensates. Second, the leaf-wise LightGBM implementation is more computationally efficient for repeated tuning [Ke et al., 2017]. Third, boosted trees capture nonlinear interactions within the hybrid sparse feature space (word and character TF-IDF plus domain indicators) more effectively than a bagged forest. For severely imbalanced classes and sparse text features, this expressiveness gap is critical.

4.1.3 BPNN & MLP

The MLP achieved only 0.5749 on the public leaderboard. In our previous work [Huang et al., 2022], BPNN was a strong candidate because the feature space was lower-dimensional, structured, and the class imbalance was less extreme. In the current brain-tumor problem, the input is a highly sparse and heterogeneous matrix (one-hot tabular variables, engineered indicators, and dual TF-IDF views). Neural networks must learn weights for thousands of sparse coordinates, which raises estimation variance and encourages overfitting at this dataset scale [Goodfellow et al., 2016]. Furthermore, with minority classes having only a handful of examples, stochastic gradient updates are dominated by majority classes, causing the network to collapse. Tree models can isolate a

minority pattern through a few highly informative splits on key tokens (e.g., "pineal" or "suprasellar"), whereas an MLP struggles to reliably estimate signal distributed across many weights.

4.1.4 ResNet Image Features

Integrating ResNet features (8192 dimensions) lowered validation Macro F_1 from 0.7432 to 0.7251. The extracted features lacked sufficient discriminative power for five tumor subtypes. Adding 8,192 dense features also increased model capacity, exacerbating overfitting.

4.1.5 Bio_ClinicalBERT

BERT failed not because of implementation difficulty, but because the task is too simple. Our classification primarily relies on detecting specific keywords ("pineal", "sellar", "enhancement", "edema") in 120-180 token reports. BERT's complex attention mechanism is unnecessary; it may even dilute localized keyword signals. TF-IDF's bag-of-words representation naturally preserves keyword identity as distinct features. For this task, BERT is overcomplicated and underperforms.

4.2 Why do we add feature augmentation?

Feature augmentation—specifically the engineering of domain-informed binary indicators and text statistics discussed in Section 2.3—is introduced primarily to combat the severe class imbalance present in the dataset. While TF-IDF vectorization effectively captures the broad semantic landscape of the radiology reports, purely learned representations can struggle to isolate stable decision boundaries for extreme minority classes, such as Pineal/Choroid plexus and Sellar-region tumors, which have very few training examples.

By explicitly injecting medical domain knowledge through deterministic rules (e.g., flagging the presence of "pineal" or "suprasellar" keywords, or defining interaction terms like enhancement $\times$ edema), we provide the LightGBM model with low-variance, high-signal discriminators. This allows the gradient boosting algorithm to make direct, clean splits for minority classes without having to discover complex combinations of sparse TF-IDF tokens from limited data. Furthermore, macroscopic structural features, such as report length and unique-word ratios, capture the overall complexity and detail of the clinical narrative—patterns that n-grams inherently miss. Ultimately, this targeted augmentation stabilizes the learning process and ensures that under-represented classes receive adequate probability mass during inference.

4.3 Overall assessment of our model

Our LightGBM pipeline stands as a robust and interpretable baseline for this clinical task. By combining dual-view TF-IDF representations with domain-informed engineered indicators and ANOVA-filtered radiomics, the model effectively leverages the

heterogeneous data sources available. The targeted upsampling and posterior class-scale tuning successfully mitigate the extreme class imbalance, preserving predictive sensitivity for minority classes (such as Pineal/Choroid plexus and Sellar-region tumors) without collapsing to majority-class predictions.

However, the primary limitation remains the extreme scarcity of training examples for the minority classes. Because these classes have such low support, per-class metrics and decision boundaries can still exhibit statistical instability. Moving forward, the most impactful improvements will likely stem from acquiring additional data for these under-represented tumor subtypes, rather than deploying increasingly complex architectures.

5 Conclusion

This project successfully developed a high-performing multiclass classification pipeline for primary brain and CNS tumors, achieving a robust balance between predictive power and computational efficiency. We demonstrated that gradient-boosted trees, when paired with thoughtful feature engineering, missingness handling, and Bayesian hyperparameter optimization, can effectively synthesize sparse textual features with structured clinical and radiomic data.

The careful handling of class imbalance—via upsampling and probability scaling—proved crucial in maintaining diagnostic sensitivity for extremely rare tumor subtypes. While advanced neural architectures and deep embeddings (like MLP and BERT) struggled with the dataset's sparsity and imbalance, our interpretable LightGBM approach achieved strong and stable validation performance, specifically a macro-F_1 of 0.81 and a weighted-F_1 of 0.88. Ultimately, this work highlights the value of domain-informed feature extraction and principled optimization in medical machine learning applications where data is heterogeneous and severely imbalanced.

References

[Breiman, 2001] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

[Gillies et al., 2016] Gillies, R. J., Kinahan, P. E., and Hricak, H. (2016). Radiomics: Images are more than pictures, they are data. *Radiology*, 278(2):563–577.

[Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

[Huang et al., 2022] Huang, G., Cheng, A., and Gao, Y. (2022). Machine learning improvements to the accuracy of predicting specific language impairment. In *2022 International Conference on Image Processing, Computer Vision and Machine Learning (ICICML)*. IEEE.

[Ke et al., 2017] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 3146–3154.

[Pedregosa et al., 2011] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.

[Price et al., 2024] Price, M., Barnholtz-Sloan, J. S., Ostrom, Q. T., et al. (2024). Cbtrus statistical report: Primary brain and other central nervous system tumors diagnosed in the united states in 2017–2021. *Neuro-Oncology*, 26(Suppl 5). CBTRUS Statistical Report.

[Ramos, 2003] Ramos, J. (2003). Using tf-idf to determine word relevance in document queries. Proceedings of the First Instructional Conference on Machine Learning.

[Snoek et al., 2012] Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems 25 (NeurIPS 2012)*.

[Wang et al., 2024] Wang, Y.-R. J. et al. (2024). Advancing presurgical non-invasive molecular subgroup prediction in medulloblastoma using artificial intelligence and mri signatures. *Cancer Cell*, 42:1239–1257.

[Zwanenburg et al., 2020] Zwanenburg, A., Vallieres, M., Abdalah, M. A., et al. (2020). The image biomarker standardization initiative: Standardized quantitative radiomics for high-throughput image-based phenotyping. *Radiology*, 295:328–338.

Appendix

A. Code for Section 4.1 Alternatives

Due to length constraints, the full implementation code for the alternative models and feature selection strategies discussed in Section 4.1 (Forward Stepwise Selection, Random Forest, MLP/BPNN, ResNet integration, and Bio_ClinicalBERT) is available in our public GitHub repository: here

B. Utility Functions and HPO-integrated LightGBM Workflow

The auxiliary utility functions (`utils.py`) and the Bayesian Hyperparameter Optimization workflow (`lgbm.py`) are omitted here for brevity. They can be accessed in our public GitHub repository at: here

C. Fixed-Parameter LightGBM (`lgbm_fixed.py`)

```
import json
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from lightgbm import LGBMClassifier
from scipy.sparse import csr_matrix, hstack, vstack
from sklearn.exceptions import ConvergenceWarning
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_selection import f_classif
from sklearn.metrics import accuracy_score, classification_report,
   f1_score, recall_score, roc_curve, auc
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import LabelEncoder, label_binarize
from utils import build_submission

warnings.filterwarnings(
    "ignore",
    message="X does not have valid feature names, but LGBMClassifier
    was fitted with feature names",
    category=UserWarning,
)
warnings.filterwarnings(
    "ignore",
```

```
    message="The optimal value found for dimension .* of parameter
   k1__k2__length_scale is close to the specified upper bound
   100.0.*",
    category=ConvergenceWarning,
    module="sklearn.gaussian_process.kernels",
)

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
CLINICAL_DIR = DATA_DIR / "clinical_information"
TRAIN_RADIOMICS_DIR = DATA_DIR / "radiomics_info" / "train"
VAL_RADIOMICS_DIR = DATA_DIR / "radiomics_info" / "val"
TEST_RADIOMICS_DIR = DATA_DIR / "radiomics_info" / "test"

FOCUS_CLASSES = [
    "Brain Metastase Tumour",
    "Pineal tumour and Choroid plexus tumour",
    "Tumors of the sellar region",
]

FIXED_PARAMS = {
    "n_estimators": 574,
    "learning_rate": 0.16151497813164442,
    "num_leaves": 43,
    "max_depth": 13,
    "min_child_samples": 25,
    "subsample": 0.8686242756333769,
    "colsample_bytree": 0.9578829278425386,
    "reg_alpha": 2.6062996462909116e-05,
    "reg_lambda": 0.11500730663980376,
    "objective": "multiclass",
    "class_weight": "balanced",
    "random_state": 42,
    "n_jobs": -1,
    "verbosity": -1,
}
FIXED_CLASS_SCALES = {
    'Brain Metastase Tumour': 1.0,
    'Pineal tumour and Choroid plexus tumour': 3.0,
    'Tumors of the sellar region': 1.9000000000000001
}

KFOLD_N_SPLITS = 5
KFOLD_RANDOM_STATE = 42
OOF_SCALE_N_SPLITS = 5
```

```python
def load_json_split(name: str):
    with (DATA_DIR / f"{name}.json").open("r") as f:
        return json.load(f)


def load_optional_clinical_csv(name: str):
    csv_path = CLINICAL_DIR / f"{name}_patient_info.csv"
    if csv_path.exists():
        return pd.read_csv(csv_path)
    return None


def load_optional_radiomics_split(split_dir: str | Path, split_name:
    str):
    p = Path(split_dir)
    if not p.exists():
        return None
    files = sorted(p.glob(f"*_radiomics_{split_name}.csv"))
    if not files:
        return None

    merged = None
    for fp in files:
        modality = fp.name.replace(f"_radiomics_{split_name}.csv", "
   ")
        df = pd.read_csv(fp, encoding="utf-8-sig")
        if "case_id" not in df.columns:
            continue
        df = df.drop(columns=["sex", "age", "modality"], errors="
   ignore")
        rad_cols = [c for c in df.columns if c.startswith("rad_")]
        if not rad_cols:
            continue
        keep = df[["case_id"] + rad_cols].copy()
        keep["case_id"] = pd.to_numeric(keep["case_id"], errors="
   coerce")
        keep = keep.dropna(subset=["case_id"])
        keep["case_id"] = keep["case_id"].astype(int)
        keep = keep.rename(columns={c: f"{modality}__{c}" for c in
   rad_cols})
        merged = keep if merged is None else pd.merge(merged, keep,
   on="case_id", how="outer")
    return merged


def select_radiomics_by_anova(train_df: pd.DataFrame, p_threshold:
   float = 0.05):
```

```
    radiomics_cols = [c for c in train_df.columns if "__rad_" in c
   or c.startswith("rad_")]
    if not radiomics_cols:
        return []

    y = train_df["Overall_class"]
    valid_mask = y.notna()
    if int(valid_mask.sum()) == 0 or int(y[valid_mask].nunique()) <
   2:
        return radiomics_cols

    X = train_df.loc[valid_mask, radiomics_cols].apply(pd.to_numeric,
    errors="coerce")
    X = X.fillna(X.median())
    y = y.loc[valid_mask]
    _, pvals = f_classif(X, y)
    return [
        c for c, p in zip(radiomics_cols, pvals)
        if np.isfinite(p) and float(p) <= float(p_threshold)
    ]


def json_to_df(data):
    rows = []
    for case_id, info in data.items():
        report = info.get("report", "")
        modalities = info.get("available_modalities", [])
        label = info.get("Overall_class", None)
        rows.append(
            {
                "case_id": int(case_id),
                "Overall_class": label,
                "report": report,
                "n_modalities": len(modalities),
            }
        )
    return pd.DataFrame(rows)


def combine_split(clinical_df: pd.DataFrame | None, json_df: pd.
   DataFrame):
    if clinical_df is None:
        return json_df.copy()
    return pd.merge(clinical_df, json_df, on="case_id", how="left")
```

```python
def align_columns_from_train(train_df: pd.DataFrame, val_df: pd.
    DataFrame, test_df: pd.DataFrame):
    for c in train_df.columns:
        if c not in val_df.columns:
            val_df[c] = np.nan
        if c not in test_df.columns:
            test_df[c] = np.nan
    return train_df, val_df, test_df


def add_report_features(df: pd.DataFrame):
    df = df.copy()
    df["report"] = df["report"].fillna("").astype(str).str.lower()
    report_tokens = df["report"].str.findall(r"\b\w+\b")

    df["report_len"] = df["report"].apply(len)
    df["report_word_count"] = report_tokens.apply(len)
    df["report_unique_word_count"] = report_tokens.apply(lambda toks:
    len(set(toks)))
    df["report_unique_ratio"] = (
        df["report_unique_word_count"] / df["report_word_count"].
    clip(lower=1)
    )
    df["report_digit_count"] = df["report"].str.count(r"\d")
    df["report_punct_count"] = df["report"].str.count(r"[^\w\s]")
    df["has_enhancement"] = df["report"].str.contains("enhancement").
    astype(int)
    df["has_edema"] = df["report"].str.contains("edema").astype(int)
    df["has_midline_shift"] = df["report"].str.contains("midline").
    astype(int)
    df["has_hydrocephalus"] = df["report"].str.contains("
    hydrocephalus").astype(int)
    df["has_multiple"] = df["report"].str.contains("multiple").
    astype(int)
    df["has_diffusion_restrict"] = df["report"].str.contains(r"
    diffusion|restrict").astype(int)
    df["has_cystic"] = df["report"].str.contains(r"cyst|cystic").
    astype(int)
    df["has_extra_axial_meningioma_keywords"] = df["report"].str.
    contains(
        r"dural tail|extra-axial|parasagittal|convexity|falx|
    meningioma"
    ).astype(int)
    df["has_pineal"] = df["report"].str.contains(r"pineal").astype(
    int)
    df["has_sellar"] = df["report"].str.contains(r"sellar|
    suprasellar|pituitary").astype(int)
```

```
    df["has_ventricular"] = df["report"].str.contains(r"ventricle|
    ventricular").astype(int)
    df["focus_keyword_hits"] = (
        df["has_pineal"] + df["has_sellar"] + df["has_hydrocephalus"
    ] + df["has_ventricular"]
    )
    df["enhancement_x_edema"] = df["has_enhancement"] * df["
    has_edema"]
    df["hydrocephalus_x_ventricular"] = df["has_hydrocephalus"] * df
    ["has_ventricular"]
    return df


def fill_missing(train: pd.DataFrame, val: pd.DataFrame, test: pd.
    DataFrame):
    for col in train.columns:
        if col == "Overall_class":
            continue
        if col == "report":
            train[col] = train[col].fillna("")
            val[col] = val[col].fillna("")
            test[col] = test[col].fillna("")
            continue
        if train[col].dtype == "object" or pd.api.types.
    is_string_dtype(train[col]):
            train[col] = train[col].fillna("Unknown")
            val[col] = val[col].fillna("Unknown")
            test[col] = test[col].fillna("Unknown")
        elif pd.api.types.is_numeric_dtype(train[col]):
            median_value = train[col].median()
            train[col] = train[col].fillna(median_value)
            val[col] = val[col].fillna(median_value)
            test[col] = test[col].fillna(median_value)
        else:
            train[col] = train[col].fillna("Unknown")
            val[col] = val[col].fillna("Unknown")
            test[col] = test[col].fillna("Unknown")
    return train, val, test


def build_features(X_train: pd.DataFrame, X_val: pd.DataFrame,
    X_test: pd.DataFrame):
    tabular_cols = [c for c in X_train.columns if c not in ["report",
     "case_id"]]
    X_train_tab = X_train[tabular_cols].copy()
    X_val_tab = X_val[tabular_cols].copy()
    X_test_tab = X_test[tabular_cols].copy()
```

```
all_tab = pd.concat([X_train_tab, X_val_tab, X_test_tab], axis
=0)
all_tab = pd.get_dummies(all_tab, drop_first=False)
X_train_tab_encoded = all_tab.iloc[: len(X_train_tab), :]
X_val_tab_encoded = all_tab.iloc[len(X_train_tab): len(
X_train_tab) + len(X_val_tab), :]
X_test_tab_encoded = all_tab.iloc[len(X_train_tab) + len(
X_val_tab):, :]

tfidf_word = TfidfVectorizer(ngram_range=(1, 2), min_df=2,
max_features=2500)
tfidf_char = TfidfVectorizer(
    analyzer="char_wb",
    ngram_range=(3, 5),
    min_df=2,
    max_features=1200,
)

X_train_text_word = tfidf_word.fit_transform(X_train["report"])
X_val_text_word = tfidf_word.transform(X_val["report"])
X_test_text_word = tfidf_word.transform(X_test["report"])
X_train_text_char = tfidf_char.fit_transform(X_train["report"])
X_val_text_char = tfidf_char.transform(X_val["report"])
X_test_text_char = tfidf_char.transform(X_test["report"])
X_train_text = hstack([X_train_text_word, X_train_text_char],
format="csr")
X_val_text = hstack([X_val_text_word, X_val_text_char], format="
csr")
X_test_text = hstack([X_test_text_word, X_test_text_char],
format="csr")

X_train_encoded = hstack(
    [csr_matrix(X_train_tab_encoded.to_numpy(dtype="float32")),
X_train_text],
    format="csr",
)
X_val_encoded = hstack(
    [csr_matrix(X_val_tab_encoded.to_numpy(dtype="float32")),
X_val_text],
    format="csr",
)
X_test_encoded = hstack(
    [csr_matrix(X_test_tab_encoded.to_numpy(dtype="float32")),
X_test_text],
    format="csr",
)
```

```
    return X_train_encoded, X_val_encoded, X_test_encoded


def to_lgbm_input(X):
    if isinstance(X, pd.DataFrame):
        return X.to_numpy(dtype=np.float32, copy=False)
    if hasattr(X, "astype"):
        return X.astype(np.float32, copy=False)
    return X


def blend_score(y_true, y_pred, focus_ids):
    micro = float(f1_score(y_true, y_pred, average="micro"))
    weighted = float(f1_score(y_true, y_pred, average="weighted"))
    if len(focus_ids) == 0:
        focus_recall = weighted
    else:
        recalls = recall_score(y_true, y_pred, labels=focus_ids,
   average=None, zero_division=0)
        focus_recall = float(np.mean(recalls))
    return 0.45 * micro + 0.35 * weighted + 0.20 * focus_recall


def upsample_focus_classes(X, y, focus_ids, seed=42):
    if len(focus_ids) == 0:
        return X, y
    rng = np.random.default_rng(seed)
    y = np.asarray(y, dtype=np.int64)
    class_counts = np.bincount(y)
    median_count = int(np.median(class_counts[class_counts > 0]))
    x_blocks = [X]
    y_blocks = [y]
    for cls_id in focus_ids:
        cls_idx = np.where(y == cls_id)[0]
        if len(cls_idx) == 0:
            continue
        target_count = max(len(cls_idx) * 3, median_count)
        extra = target_count - len(cls_idx)
        if extra <= 0:
            continue
        sampled = rng.choice(cls_idx, size=extra, replace=True)
        x_blocks.append(X[sampled])
        y_blocks.append(y[sampled])
    X_aug = vstack(x_blocks, format="csr")
    y_aug = np.concatenate(y_blocks)
    perm = rng.permutation(len(y_aug))
    return X_aug[perm], y_aug[perm]
```

```python
def apply_class_scales(proba, scales):
    adjusted = proba * scales.reshape(1, -1)
    denom = adjusted.sum(axis=1, keepdims=True)
    denom = np.where(denom <= 0.0, 1.0, denom)
    return adjusted / denom


def tune_class_scales(y_true, proba, focus_ids, rounds=4):
    n_classes = proba.shape[1]
    if len(focus_ids) == 0:
        return np.ones(n_classes, dtype=np.float64), blend_score(
   y_true, np.argmax(proba, axis=1), focus_ids)

    scales = np.ones(n_classes, dtype=np.float64)
    best_score = blend_score(y_true, np.argmax(proba, axis=1),
   focus_ids)
    grid = np.unique(np.concatenate([np.linspace(0.5, 3.0, 26), np.
   array([1.15, 1.35, 1.65, 2.25, 2.75])]))

    for _ in range(rounds):
        improved = False
        for cls_id in focus_ids:
            best_local = scales[cls_id]
            for g in grid:
                trial = scales.copy()
                trial[cls_id] = g
                pred = np.argmax(apply_class_scales(proba, trial),
   axis=1)
                score = blend_score(y_true, pred, focus_ids)
                if score > best_score:
                    best_score = score
                    best_local = g
                    improved = True
            scales[cls_id] = best_local
        if not improved:
            break
    return scales, best_score


def run_kfold_cv(X_all: pd.DataFrame, y_all: pd.Series, X_test_ref:
   pd.DataFrame, n_splits=5, seed=42):
    y_all = y_all.reset_index(drop=True)
    X_all = X_all.reset_index(drop=True)

    label_encoder_cv = LabelEncoder()
```

```
label_encoder_cv.fit(y_all)
y_all_enc = label_encoder_cv.transform(y_all)
focus_ids_cv = [
    int(label_encoder_cv.transform([c])[0])
    for c in FOCUS_CLASSES if c in label_encoder_cv.classes_
]

skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
random_state=seed)
fold_scores = []

for fold_idx, (tr_idx, va_idx) in enumerate(skf.split(X_all,
y_all_enc), start=1):
    X_tr_fold = X_all.iloc[tr_idx].copy()
    X_va_fold = X_all.iloc[va_idx].copy()
    y_tr_fold = y_all.iloc[tr_idx]
    y_va_fold = y_all.iloc[va_idx]

    X_tr_enc, X_va_enc, _ = build_features(X_tr_fold, X_va_fold,
 X_test_ref)
    y_tr_enc = label_encoder_cv.transform(y_tr_fold)
    y_va_enc = label_encoder_cv.transform(y_va_fold)

    X_tr_bal, y_tr_bal = upsample_focus_classes(X_tr_enc,
y_tr_enc, focus_ids_cv, seed=seed + fold_idx)

    fold_params = FIXED_PARAMS.copy()

    model = LGBMClassifier(**fold_params)
    model.fit(to_lgbm_input(X_tr_bal), y_tr_bal)

    va_proba = model.predict_proba(to_lgbm_input(X_va_enc))
    fold_scales, _ = tune_class_scales(y_va_enc, va_proba,
focus_ids_cv, rounds=4)
    va_pred = np.argmax(apply_class_scales(va_proba, fold_scales
), axis=1)
    fold_blended = blend_score(y_va_enc, va_pred, focus_ids_cv)
    fold_scores.append(float(fold_blended))

    print(f"[CV] Fold {fold_idx}/{n_splits} blended score: {
fold_blended:.6f}")

if fold_scores:
    print(f"[CV] Mean blended score: {float(np.mean(fold_scores)
):.6f}")
    print(f"[CV] Std blended score: {float(np.std(fold_scores))
:.6f}")
```

```
    return fold_scores


def tune_class_scales_from_oof(
    X_all: pd.DataFrame,
    y_all: pd.Series,
    X_test_ref: pd.DataFrame,
    best_params: dict,
    focus_ids: list[int],
    n_splits=5,
    seed=42,
):
    y_all = y_all.reset_index(drop=True)
    X_all = X_all.reset_index(drop=True)
    y_all_enc = LabelEncoder().fit_transform(y_all)
    n_classes = int(np.max(y_all_enc)) + 1
    oof_proba = np.zeros((len(X_all), n_classes), dtype=np.float64)
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
   random_state=seed)

    for fold_idx, (tr_idx, va_idx) in enumerate(skf.split(X_all,
   y_all_enc), start=1):
        X_tr_fold = X_all.iloc[tr_idx].copy()
        X_va_fold = X_all.iloc[va_idx].copy()
        y_tr_fold_enc = y_all_enc[tr_idx]

        X_tr_enc, X_va_enc, _ = build_features(X_tr_fold, X_va_fold,
     X_test_ref)
        X_tr_bal, y_tr_bal = upsample_focus_classes(X_tr_enc,
   y_tr_fold_enc, focus_ids, seed=seed + fold_idx)

        model = LGBMClassifier(**best_params)
        model.fit(to_lgbm_input(X_tr_bal), y_tr_bal)
        oof_proba[va_idx] = model.predict_proba(to_lgbm_input(
   X_va_enc))

    scales, score = tune_class_scales(y_all_enc, oof_proba,
   focus_ids, rounds=4)
    return scales, float(score)


def save_multiclass_roc_plot(y_true_enc: np.ndarray, proba: np.
   ndarray, class_names: list[str], out_path: Path):
    y_true_enc = np.asarray(y_true_enc, dtype=np.int64)
    n_classes = proba.shape[1]
```

```
    y_true_bin = label_binarize(y_true_enc, classes=np.arange(
    n_classes))

    fpr = {}
    tpr = {}
    roc_auc = {}
    valid_classes = []
    for i in range(n_classes):
        y_bin = y_true_bin[:, i]
        # roc_curve requires both positive and negative samples.
        if np.unique(y_bin).size < 2:
            continue
        fpr[i], tpr[i], _ = roc_curve(y_bin, proba[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])
        valid_classes.append(i)

    out_path.parent.mkdir(parents=True, exist_ok=True)
    plt.figure(figsize=(9, 7))
    for i in valid_classes:
        name = class_names[i] if i < len(class_names) else f"Class {
    i}"
        plt.plot(fpr[i], tpr[i], lw=2, label=f"{name} (AUC={roc_auc[
    i]:.3f})")
    plt.plot([0, 1], [0, 1], "k--", lw=1, label="Random baseline")
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("Validation ROC Curves (Fixed LGBM)")
    plt.legend(loc="lower right", fontsize=8, frameon=False)
    plt.tight_layout()
    plt.savefig(out_path, dpi=220)
    plt.close()
    return {class_names[i]: float(roc_auc[i]) for i in valid_classes
    if i < len(class_names)}


def main():
    start_time = time.time()

    train_csv = load_optional_clinical_csv("train")
    val_csv = load_optional_clinical_csv("val")
    test_csv = load_optional_clinical_csv("test")

    train_json = load_json_split("train")
    val_json = load_json_split("val")
    test_json = load_json_split("test")
```

```
train_df = combine_split(train_csv, json_to_df(train_json))
val_df = combine_split(val_csv, json_to_df(val_json))
test_df = combine_split(test_csv, json_to_df(test_json))

# Optional radiomics merge + ANOVA filtering (drop p > 0.05).
train_rad = load_optional_radiomics_split(TRAIN_RADIOMICS_DIR, "
train")
val_rad = load_optional_radiomics_split(VAL_RADIOMICS_DIR, "val"
)
test_rad = load_optional_radiomics_split(TEST_RADIOMICS_DIR, "
test")
if train_rad is not None:
    train_df = pd.merge(train_df, train_rad, on="case_id", how="
left")
if val_rad is not None:
    val_df = pd.merge(val_df, val_rad, on="case_id", how="left")
if test_rad is not None:
    test_df = pd.merge(test_df, test_rad, on="case_id", how="
left")

selected_radiomics = select_radiomics_by_anova(train_df,
p_threshold=0.05)
all_radiomics = [c for c in train_df.columns if "__rad_" in c or
 c.startswith("rad_")]
drop_radiomics = [c for c in all_radiomics if c not in set(
selected_radiomics)]
if drop_radiomics:
    train_df = train_df.drop(columns=drop_radiomics, errors="
ignore")
    val_df = val_df.drop(columns=drop_radiomics, errors="ignore"
)
    test_df = test_df.drop(columns=drop_radiomics, errors="
ignore")
if all_radiomics:
    print(
        f"Radiomics ANOVA filter: kept {len(selected_radiomics)
}/{len(all_radiomics)} features "
        f"(p <= 0.05)."
    )

# Missingness policy for retained radiomics.
RADIOMICS_MAX_MISSING_RATE = 0.40
if selected_radiomics:
    too_sparse = [
        c for c in selected_radiomics
```

```
        if float(train_df[c].isna().mean()) >
RADIOMICS_MAX_MISSING_RATE
    ]
    if too_sparse:
        train_df = train_df.drop(columns=too_sparse, errors="
ignore")
        val_df = val_df.drop(columns=too_sparse, errors="ignore"
)
        test_df = test_df.drop(columns=too_sparse, errors="
ignore")
        selected_radiomics = [c for c in selected_radiomics if c
 not in set(too_sparse)]
        print(
            f"Radiomics missingness filter: dropped {len(
too_sparse)} columns "
            f"(missing > {RADIOMICS_MAX_MISSING_RATE:.0%})."
        )

    for c in selected_radiomics:
        if float(train_df[c].isna().mean()) > 0.10:
            if c not in val_df.columns:
                val_df[c] = np.nan
            if c not in test_df.columns:
                test_df[c] = np.nan
            ind = f"{c}__is_missing"
            train_df[ind] = train_df[c].isna().astype(int)
            val_df[ind] = val_df[c].isna().astype(int)
            test_df[ind] = test_df[c].isna().astype(int)

train_df, val_df, test_df = align_columns_from_train(train_df,
val_df, test_df)

train = add_report_features(train_df)
val = add_report_features(val_df)
test = add_report_features(test_df)

train = train.drop(columns=["Sex"], errors="ignore")
val = val.drop(columns=["Sex"], errors="ignore")
test = test.drop(columns=["Sex"], errors="ignore")
train, val, test = fill_missing(train, val, test)

X_train = train.drop(columns=["Overall_class"])
y_train = train["Overall_class"]
X_val = val.drop(columns=["Overall_class"])
y_val = val["Overall_class"]
X_test = test.copy()
```

```
X_train_encoded, X_val_encoded, _ = build_features(X_train,
X_val, X_test)

label_encoder = LabelEncoder()
label_encoder.fit(pd.concat([y_train, y_val], axis=0))
y_train_enc = label_encoder.transform(y_train)
y_val_enc = label_encoder.transform(y_val)
focus_ids = [int(label_encoder.transform([c])[0]) for c in
FOCUS_CLASSES if c in label_encoder.classes_]

X_train_balanced, y_train_balanced = upsample_focus_classes(
    X_train_encoded, y_train_enc, focus_ids, seed=42
)

best_params = FIXED_PARAMS.copy()

lgbm = LGBMClassifier(**best_params)
lgbm.fit(to_lgbm_input(X_train_balanced), y_train_balanced)

val_proba = lgbm.predict_proba(to_lgbm_input(X_val_encoded))
class_scales = np.ones(len(label_encoder.classes_), dtype=np.
float64)
for cls_name, scale in FIXED_CLASS_SCALES.items():
    if cls_name in label_encoder.classes_:
        cls_id = int(label_encoder.transform([cls_name])[0])
        class_scales[cls_id] = float(scale)
val_proba_scaled = apply_class_scales(val_proba, class_scales)
roc_output_path = ROOT / "imgs" / "
lgbm_fixed_validation_roc_curve.png"
val_auc_scores = save_multiclass_roc_plot(
    y_true_enc=y_val_enc,
    proba=val_proba_scaled,
    class_names=list(label_encoder.classes_),
    out_path=roc_output_path,
)

val_pred = np.argmax(val_proba_scaled, axis=1)
val_pred_label = label_encoder.inverse_transform(val_pred)

print("Validation Accuracy:", accuracy_score(y_val,
val_pred_label))
print("Validation Micro F1:", f1_score(y_val, val_pred_label,
average="micro"))
print("Validation Macro F1:", f1_score(y_val, val_pred_label,
average="macro"))
print("Validation Weighted F1:", f1_score(y_val, val_pred_label,
 average="weighted"))
```

```
print("\nClassification Report:")
print(classification_report(y_val, val_pred_label))
print("Fixed Params:", best_params)
best_val_blended_score = float(blend_score(y_val_enc, val_pred,
focus_ids))
print("Validation Blended Score (Fixed Class Scales):",
best_val_blended_score)
print("Final Validation Strategy: LGBM Only")
best_class_scales = {cls_name: float(class_scales[i]) for i,
cls_name in enumerate(label_encoder.classes_)}
print("Best Class Scales:", best_class_scales)
print("Validation ROC AUC by class:", val_auc_scores)
print("Validation ROC curve saved to:", roc_output_path)

X_full = pd.concat([X_train, X_val], axis=0)
y_full = pd.concat([y_train, y_val], axis=0)
X_full_tab = X_full.drop(columns=["report", "case_id"], errors="
ignore")
X_test_tab_final = X_test.drop(columns=["report", "case_id"],
errors="ignore")
full_tab_all = pd.concat([X_full_tab, X_test_tab_final], axis=0)
full_tab_all = pd.get_dummies(full_tab_all, drop_first=False)
X_full_tab_encoded = full_tab_all.iloc[: len(X_full_tab), :]
X_test_tab_final_encoded = full_tab_all.iloc[len(X_full_tab):,
:]

tfidf_final_word = TfidfVectorizer(ngram_range=(1, 2), min_df=2,
 max_features=2500)
tfidf_final_char = TfidfVectorizer(analyzer="char_wb",
ngram_range=(3, 5), min_df=2, max_features=1200)

X_full_text_word = tfidf_final_word.fit_transform(X_full["report
"])
X_test_text_word_final = tfidf_final_word.transform(X_test["
report"])
X_full_text_char = tfidf_final_char.fit_transform(X_full["report
"])
X_test_text_char_final = tfidf_final_char.transform(X_test["
report"])
X_full_text = hstack([X_full_text_word, X_full_text_char],
format="csr")
X_test_text_final = hstack([X_test_text_word_final,
X_test_text_char_final], format="csr")

X_full_encoded = hstack([csr_matrix(X_full_tab_encoded.to_numpy(
dtype="float32")), X_full_text], format="csr")
X_test_final = hstack(
```

```
        [csr_matrix(X_test_tab_final_encoded.to_numpy(dtype="float32
    ")), X_test_text_final],
        format="csr",
    )

    y_full_enc = label_encoder.transform(y_full)
    X_full_balanced, y_full_balanced = upsample_focus_classes(
   X_full_encoded, y_full_enc, focus_ids, seed=42)

    lgbm_final = LGBMClassifier(**best_params)
    lgbm_final.fit(to_lgbm_input(X_full_balanced), y_full_balanced)

    test_proba = lgbm_final.predict_proba(to_lgbm_input(X_test_final
   ))

    test_pred = np.argmax(apply_class_scales(test_proba,
   class_scales), axis=1)
    test_pred_label = label_encoder.inverse_transform(test_pred)

    submission = build_submission(test["case_id"], test_pred_label,
   target_col="Overall_class")
    output_dir = ROOT / "submissions"
    output_dir.mkdir(parents=True, exist_ok=True)
    submission_path = output_dir / "submission.csv"
    submission.to_csv(submission_path, index=False)

    print("\nGenerated predictions:", len(submission))
    print("Submission saved to:", submission_path)
    print(f"Overall running time: {time.time() - start_time:.2f}
   seconds")


if __name__ == "__main__":
    main()
```